

Computer Graphics – Group 3 (Github Version)

For Documentation purpose and Educational Only

**Supervised by :
Dr. Mohammad Mirza**

Group Members

Student Name	ID
Saad Saeed Almalki (Team Leader)	-
Fahad Jumaan Almalki	-
Anas Sami Alharthi	-
Mohammad Abdullah Alharthi	-
Mohannad Abdullah Alhumaidi	-
Feras Awdah Almalki	-
Raed Mutheb Alsubaie	-
Abdulrahman Muqbil Alharthi	-
Osama Nayaf Almutairi	-

Table of Contents :

Task / Section	Description	Page Number
Task 1	Selection and Conceptualization	Page 3
Task 2	Layout Development & Individual Contributions	Page 4
Task 3	Scene Assembly and Coloring	Page 8
Task 4	Interactive System (Keyboard & Mouse)	Page 10
Task 5	Scene Enhancement & Animation (Bonus)	Page 12
Task 6	Presentation Video	Page 13

Task 1

1. Chosen Landmark

Landmark Name: College of Computers and Information Technology (CIT Building) at Taif University.

2. Design Approach

Approach: Stylized and Creative 2D Design.

3. Descriptive Paragraph

For our computer graphics project, we decided to recreate the CIT College Building here at Taif University. We went with a stylized and clean 2D creative approach using standard OpenGL primitives instead of a complex realistic look. The main idea was to capture the essential architecture of our college, so we built the front facade using simple quads (GL_QUADS) for the main walls, entrance, and windows. We also added a custom text sign at the top that says 'CIT' using stroke fonts to make it recognizable. To make the scene feel alive, we used a flexible color function that easily changes the theme between day and night when the user interacts with the program, adjusting the sky, adding moving clouds, and switching on the window lights. We made sure every single function relies entirely on our course syllabus, focusing on clear code organization and proper handling of 2D transformations

4. Concept Sketch

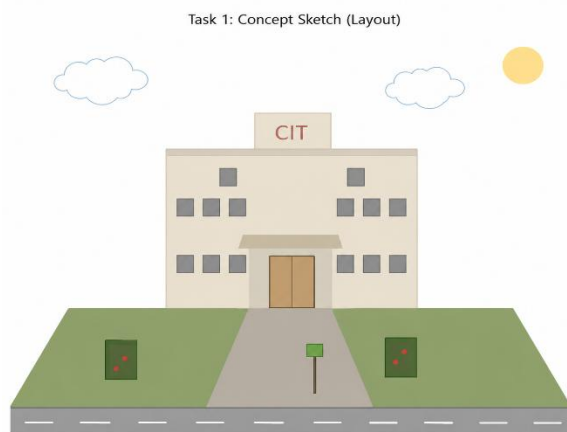
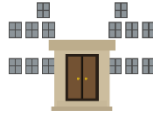


Figure 1: Conceptual layout sketch of the CIT college building scene during daytime.

Task 2

Student Name	Student ID	Assigned Object / Module	Implementation Strategy (How it was built)
Anas Sami Al-Harthi	-	Clouds & Sky Elements 	Built the clouds using standard <code>GL_TRIANGLE_FAN</code> circles, and managed the sky background colors using conditional statements that automatically shift between day and night themes during user interaction.
Muhanad Abdullah Al-Humaidi	-	Windows and Main Entrance Door 	Constructed the main entrance and window modules using <code>GL_QUADS</code> for the frames, glass panels, and doors. I used <code>GL_TRIANGLE_FAN</code> for the circular door handles. The <code>drawWindow()</code> function uses <code>(x, y)</code> parameters to easily duplicate and position the windows across the building.
Abdulrahman Moqbel Al-Harthi	-	Building Structure and Main Facade 	Constructed the main building walls and the front facade using <code>GL_QUADS</code> . The <code>drawWalls()</code> function uses direct vertex coordinates (<code>glVertex2f</code>) to shape the side walls, while the <code>drawFacade()</code> function utilizes a custom rectangle method. Colors were applied using a dynamic color function to support the scene's day

Student Name	Student ID	Assigned Object / Module	Implementation Strategy (How it was built)
			and night theme transitions.
Saad Saeed Almalki	-	Rooftop logo of college 'CIT' & Stroke-Font Text 	Built the rooftop signage that sits on top of the facade. The signboard panel and its two support posts were constructed with GL_QUADS, while the <code>drawCITSign(x, y)</code> function renders the 'CIT' logo using GLUT stroke fonts via <code>glutStrokeCharacter()</code> . The text is wrapped inside <code>glPushMatrix()</code> / <code>glPopMatrix()</code> and is positioned and scaled with <code>glTranslatef()</code> and <code>glScalef()</code> . So the sign can be moved or resized independently of the rest of the scene. Its color is taken from the shared day and night color function so the sign always matches the building theme.
Mohammad Abdullah AlHarthi	-	Ground Plaza & Side Pathways 	Built the open ground area in front of the building. The main plaza and the two angled side pathways were drawn as GL_POLYGON shapes with explicit <code>glVertex2f()</code> vertices, so the ground meets the base of the walls cleanly. The <code>drawFloor()</code> function colors every section through the shared <code>col()</code> function, so the paving darkens at night with the rest of the scene.
Feras Awdah Almalki	-	Road & Lane Markings 	Built the road running across the front of the scene. The asphalt, the dark lane, and the curb

Student Name	Student ID	Assigned Object / Module	Implementation Strategy (How it was built)
			strip were drawn with <code>GL_QUADS</code> through the <code>rect()</code> helper, and the dashed center line was produced with a <code>for</code> loop that repeats small white rectangles at a fixed spacing. The <code>drawRoad()</code> function uses <code>col()</code> so the road tone shifts between day and night .
Raed Mutheb Alsubaie	-	Bushes 	Built the green bushes of the college garden. Each bush is produced by the reusable <code>drawBush(x, y, w, h)</code> function, which layers several <code>GL_QUADS</code> for the frame and inner panels and adds a <code>disc()</code> bolt detail. Because the function is parameterized by position and size, <code>drawBushes()</code> draws two differently sized bushes just by calling it with different arguments .
Osama Nayaf Almutairi	-	Roadside Signpost 	Built the roadside signpost near the entrance. The metal pole and the green sign panel were constructed with <code>GL_QUADS</code> via <code>rect()</code> , and the panel brightness is driven by a <code>sinf()</code> value so it can pulse and glow when toggled. The <code>drawSignPost()</code> function reads the shared day and night state through <code>col()</code> so the post matches the scene lighting.
Fahad Jum'aan Almalki	-	Animated Man	Built the human figure that moves in front of the building . The legs and body were made from <code>GL_QUADS</code> and the head

Student Name	Student ID	Assigned Object / Module	Implementation Strategy (How it was built)
			from a <code>GL_TRIANGLE_FAN</code> <code>disc()</code> . The whole figure is drawn around a movable position passed into <code>drawPerson(x)</code> , and a small <code>sinf()</code> offset on the height gives it a gentle bobbing motion as it walks.

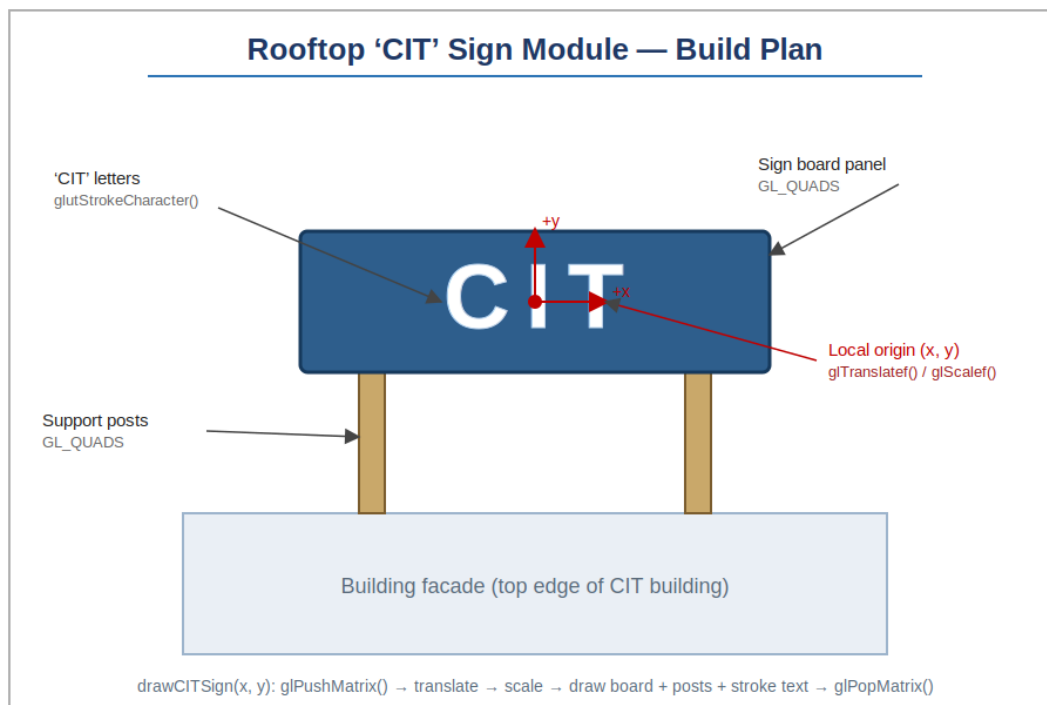


Figure 2: Proposed build plan for the rooftop 'CIT' sign module — the sign board and support posts drawn with `GL_QUADS`, the 'CIT' letters rendered with GLUT stroke fonts, and the whole sign positioned and scaled around a local origin using 2D transformations.

Task 3

1. Assembly Method

After each member finished building their own object in Task 2, all of these modules were combined into one master drawing routine, the `display()` function. Every object lives in its own function (`drawSky`, `drawWalls`, `drawWindows`, `drawEntrance`, `drawSign`, `drawFloor`, `drawRoad`, `drawBushes`, `drawSignPost`, `drawPerson`), and `display()` simply calls them in sequence . This kept the scene modular and let each member work on their part independently before everything was assembled together.

2. Drawing Order (Layering)

Since our 2D scene does not use a depth buffer, we relied on a painter's approach and drew objects from the farthest background to the nearest foreground. We start with the sky and clouds, then the building walls, facade, windows, entrance, and rooftop 'CIT' sign, followed by the ground, road, gates, signpost, and finally the animated person and the on-screen instructions. This order guarantees that nearer objects correctly overlap the ones behind them.

3. Coloring System (Day / Night)

All coloring is centralized in a single helper, `col(dr, dg, db, nr, ng, nb)`, which receives a daytime set and a night set of RGB values and chooses between them based on the global `isDaytime` flag. Every object passes its own two colors to this function instead of calling `glColor3f` directly, so the whole scene changes theme consistently from one boolean. The sky is set through the same logic, the window lights turn warm yellow at night, and the moon and stars appear only when `isDaytime` is false.

4. Descriptive Paragraph

Once every object was ready, the next step was to bring everything together into one coherent scene. We assembled all the modules inside the `display()` function and called them in a back-to-front order, so the background, building, ground, and foreground details layer correctly on top of one another. To keep the look consistent and lively, we built the coloring around a single shared color function that stores both a day and a night value for every element and switches between them through one `isDaytime` variable. This let the whole campus shift smoothly between a bright daytime view and a calm night view — changing the sky, revealing the moon and stars, drifting the clouds, and lighting up the windows — while keeping our code organized and easy to maintain, exactly in line with the goals of our course syllabus .

Student Name	Student ID	Student Work
Saad Saeed Almalki	-	For scene assembly, I was responsible for integrating the upper part of the CIT building. I positioned the rooftop sign 'CIT' so its base ($y = 83$) lines up exactly with the top band of the façade, and I made sure the call order in <code>display()</code> function places <code>drawSign()</code> <i>after</i> <code>drawFacade()</code> so the sign always renders in front of the wall instead of being hidden behind the wall. I did not change the sign geometry from Task 2; instead I reuse again the same transformation block inside <code>strokeText()</code> (<code>glPushMatrix</code> → <code>glTranslatef</code> → <code>glScalef</code>) to position and scale the letters within the full scene. For coloring, I passed the sign board its day and night values through the shared function <code>col()</code>, so the rooftop stays in the same tone as the facade in both day and night themes.
Feras Awdah Almalki	-	I handled the overall composition order and color consistency for the group. I set the back-to-front draw sequence inside <code>display()</code> — sky and clouds first, then the building (walls, facade, windows, entrance, sign), then the ground, road, bushes, signpost, and finally the

		person and the on-screen instructions — so every object layers correctly using the painter's method, since our scene uses no depth buffer. I then reviewed all modules to confirm each one routes its colors through the single <code>col(dr, dg, db, nr, ng, nb)</code> function and that the day and night values stay in the same tonal family, giving the whole scene one consistent theme. No geometry was modified in this step; my work was purely ordering the objects and unifying the coloring across everyone's modules.
--	--	--

Task 4

1-Overview

Our scene is fully interactive and responds to both the keyboard and the mouse. Although the assignment only requires two interactive features, we implemented all four options from the list: keyboard controls, mouse interaction, a day/night toggle, and on-screen instructions. All user input is handled through standard GLUT callback functions registered in `main()` — `glutKeyboardFunc(keyboard)`, `glutSpecialFunc(special)`, and `glutMouseFunc(mouse)` — and every handler calls `glutPostRedisplay()` so the window updates immediately after each interaction.

2- Keyboard Controls

The `keyboard()` and `special()` callbacks let the user control the scene directly. The man can be moved across the scene with the A / D keys or the Left / Right arrow keys, where each press changes the global `personX` value by a fixed step; the value is clamped between 8 and 92 so the figure never leaves the visible area. The main entrance door is opened and closed by pressing the SPACE key, which flips the `doorOpen` flag and redraws the door in its open or closed state.

3-Mouse Interaction

The `mouse()` callback adds a click-to-toggle feature . When the user presses the left mouse button, The program flips the `signGlow` flag , which highlights the rooftop sign 'CIT' and makes its color pulse using a `sinf()` brightness value . Clicking again turns the highlight back off, so the sign acts like a light the user can switch on and off .

4- Day / Night Toggle

The scene can switch between a day theme and a night theme at any time by pressing the N key or the Up / Down arrow keys, which invert the global isDaytime flag . Because the entire scene is colored through our shared col() function , this single boolean changes the whole atmosphere at once: the sky darkens, the sun is replaced by a moon and stars, the clouds disappear in night , and the window lights turn a yellow because the lights on .

5- On-Screen Instructions

To make the controls clear without an external manual, the drawInstructions() function prints the key bindings directly on the screen using glutBitmapCharacter() with a Helvetica bitmap font. Two lines at the top of the window summarize every control (move person, day/night, door, and sign highlight), and their text color adapts to the theme so the instructions stay readable in both day and night.

6. Descriptive Paragraph

To make the scene feel alive rather than static, we built a complete interactive system around a few simple state variables — isDaytime , doorOpen , signGlow , personX. Instead of redrawing things manually again , each input handler only changes one of these variables and then asks GLUT to refresh the screen, so the display() function always reflects the current state of the scene. This design keeps the interaction code short and easy to explain : the keyboard moves the man and toggles the door and theme , the mouse highlights the sign , and the day/night switch repaints the whole campus through our shared color function . The result is a scene the user can explore and change in real time, which fully satisfies the interactive requirement of the project .

Student Name	Student ID	Student Work
Anas Sami Al-Harthi	-	Implemented the day and night toggle. I connected the N key and the Up and Down arrow keys, inside the keyboard() and special() callbacks, to flip the global isDaytime flag. Because the whole scene is colored through the shared col() function, this single key press repaints everything at once — the sky, the window lights, and the moon and stars.
Muhanad Abdullah Al-Humaidi	-	Implemented the keyboard controls for the door and the person. The SPACE key toggles the doorOpen flag so the main entrance switches between its open and closed drawings, and the A and D

		keys (together with the Left and Right arrows) change the personX value to move the figure, which is clamped between 8 and 92 so it always stays inside the window.
Abdulrahman Moqbel Al-Harthi	-	Implemented the mouse interaction and the on-screen help. A left click in the mouse() callback toggles the signGlow flag to highlight the rooftop sign, and the drawInstructions() function prints the key bindings on screen using a bitmap font. I also registered all the input callbacks (glutKeyboardFunc, glutSpecialFunc, glutMouseFunc) in main() so the program responds to every input.

Task 5

For this task , we choosing to add a simple animation to make the scene alive and dynamic . We used already a timer() function that keeps updating a time value in code , and this drives a few small movements : the clouds in the sky , the person walk front the building , and the great sign in college garden softly pulses when it is highlighted . These animations are easy and giving us an Interactive world in our code .

Student Name	Student ID	Student Work
Raed Mutheb Alsubaie	-	Built the animation engine. I wrote the timer() function that keeps increasing the time value t and the cloudX position, re-schedules itself with glutTimerFunc(), and calls glutPostRedisplay() so the scene redraws

		continuously. This same timer drifts the clouds across the sky and resets them once they leave the screen, giving a smooth looping motion.
Osama Nayaf Almutairi	-	Added the glow and twinkle effects. The roadside signpost pulses using a <code>sinf()</code> brightness value tied to the time, so it shines softly when its highlight is switched on, and the night stars change size with another <code>sinf()</code> value to make them twinkle. Both effects reuse the same time variable updated by the timer.
Fahad Jum'aan Almalki	-	Added the movement of the person. Using a <code>sinf()</code> offset on the time value, the figure gently bobs up and down on its vertical position as it stands, which makes the walking motion look more natural. The effect is driven by the same timer, so it stays in sync with the rest of the animation.

Student works by-parts

Task 1 :

Anas Sami Al-Harthi
Mohammad Abdullah Al-Harthi

Task 2 :

All Students

Task 3 :

Saad Saeed Al-Malki
Feras Awdah Al-Malki

Task 4 :

Anas Sami Al-Harthi
Abdulrahman Moqbil Al-Harthi
Muhanad Abdullah Al-Humaidi

Task 5 :

Raed Mutheb Al-Subaie
Osama Nayaf Al-Mutairi
Fahad Jum'aan Al-Malki

Task 6 :

Anas Sami Al-Harthi (Video Presentation)
Saad Saeed Al-Malki (Coding clips)